

GaussianNB#

https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.GaussianNB.html

Description:

Gaussian Naive Bayes (GaussianNB). Can perform online updates to model parameters via `partial_fit`. For details on algorithm used to update feature means and variance online, see Stanford CS tech report STAN-CS-79-773 by Chan, Golub, and LeVeque. Read more in the User Guide. Prior probabilities of the classes. If specified, the priors are not adjusted according to the data.

Function Signature

```
class sklearn.naive_bayes.GaussianNB(*, priors=None,
var_smoothing=1e-09)[source]#
```

Parameters:

priorsarray-like of shape (n_classes,), default=None

var_smoothingfloat, default=1e-9

Attributes:

```
class sklearn.naive_bayes.GaussianNB(*, priors=None,
var_smoothing=1e-09)[source]#
```

Parameters:

priorsarray-like of shape (n_classes,), default=None

var_smoothingfloat, default=1e-9

Attributes:

```
fit(X, y, sample_weight=None)[source]#
```

Parameters:

Xarray-like of shape (n_samples, n_features)

yarray-like of shape (n_samples,)

sample_weightarray-like of shape (n_samples,), default=None

Parameters

priorsarray

Prior probabilities of the classes. If specified, the priors are not adjusted according to the data.

var_smoothingfloat, default=1e

Portion of the largest variance of all features that is added to variances for calculation stability. Added in version 0.20.

Xarray

Training vectors, where n_samples is the number of samples and n_features is the number of features.

yarray

Target values.

sample_weightarray

Weights applied to individual samples (1. for unweighted). Added in version 0.17: Gaussian Naive Bayes supports fitting with sample_weight.

paramsdict

Parameter names mapped to their values.

Xarray

Training vectors, where n_samples is the number of samples and n_features is the number of features.

yarray

Target values.

classesarray

List of all the classes that can possibly appear in the y vector. Must be provided at the first call to partial_fit, can be omitted in subsequent calls.

sample_weightarray

Weights applied to individual samples (1. for unweighted). Added in version 0.17.

Xarray

The input samples.

Xarray

The input samples.

Xarray

The input samples.

Carray

Returns the log-probability of the samples for each class in the model. The columns correspond to the classes in sorted order, as they appear in the attribute `classes_`.

Xarray

The input samples.

Carray

Returns the probability of the samples for each class in the model. The columns correspond to the classes in sorted order, as they appear in the attribute `classes_`.

Xarray

Test samples.

yarray

True labels for X.

sample_weightarray

Sample weights.

****paramsdict**

Estimator parameters.

Code Examples

```
>>> import numpy as np
>>> X = np.array([[-1, -1], [-2, -1], [-3, -2], [1, 1], [2, 1],
[3, 2]])
>>> Y = np.array([1, 1, 1, 2, 2, 2])
>>> from sklearn.naive_bayes import GaussianNB
>>> clf = GaussianNB()
>>> clf.fit(X, Y)
GaussianNB()
>>> print(clf.predict([[-0.8, -1]]))
[1]
>>> clf_pf = GaussianNB()
>>> clf_pf.partial_fit(X, Y, np.unique(Y))
GaussianNB()
>>> print(clf_pf.predict([[-0.8, -1]]))
[1]
```

Notes & Warnings

NOTE See also `BernoulliNB` Naive Bayes classifier for multivariate Bernoulli models. `CategoricalNB` Naive Bayes classifier for categorical features. `ComplementNB` Complement Naive Bayes classifier. `MultinomialNB` Naive Bayes classifier for multinomial models.

Detailed Documentation

Documentation

Gaussian Naive Bayes (GaussianNB).

Can perform online updates to model parameters via `partial_fit`. For details on algorithm used to update feature means and variance online, see Stanford CS tech report STAN-CS-79-773 by Chan, Golub, and LeVeque.

Read more in the User Guide.

Prior probabilities of the classes. If specified, the priors are not adjusted according to the data.

Portion of the largest variance of all features that is added to variances for calculation stability.

Added in version 0.20.

number of training samples observed in each class.

probability of each class.

class labels known to the classifier.

absolute additive value to variances.

Number of features seen during fit.

Added in version 0.24.

Names of features seen during fit. Defined only when X has feature names that are all strings.

Added in version 1.0.

Variance of each feature per class.

Added in version 1.0.

mean of each feature per class.

Naive Bayes classifier for multivariate Bernoulli models.

Naive Bayes classifier for categorical features.

Complement Naive Bayes classifier.

Naive Bayes classifier for multinomial models.

Fit Gaussian Naive Bayes according to X, y.

Training vectors, where `n_samples` is the number of samples and `n_features` is the number of features.

Weights applied to individual samples (1. for unweighted).

Added in version 0.17: Gaussian Naive Bayes supports fitting with `sample_weight`.

Returns the instance itself.

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

A `MetadataRequest` encapsulating routing information.

Get parameters for this estimator.

If True, will return the parameters for this estimator and contained subobjects that are estimators.

Parameter names mapped to their values.

Incremental fit on a batch of samples.

This method is expected to be called several times consecutively on different chunks of a dataset so as to implement out-of-core or online learning.

This is especially useful when the whole dataset is too big to fit in memory at once.

This method has some performance and numerical stability overhead, hence it is better to call `partial_fit` on chunks of data that are as large as possible (as long as fitting in the memory budget) to hide the overhead.

Training vectors, where `n_samples` is the number of samples and `n_features` is the number of features.

List of all the classes that can possibly appear in the `y` vector.

Must be provided at the first call to `partial_fit`, can be omitted in subsequent calls.

Weights applied to individual samples (1. for unweighted).

Added in version 0.17.

Returns the instance itself.

Perform classification on an array of test vectors `X`.

Predicted target values for `X`.

Return joint log probability estimates for the test vector X .

For each row x of X and class y , the joint log probability is given by $\log P(x, y) = \log P(y) + \log P(x|y)$, where $\log P(y)$ is the class prior probability and $\log P(x|y)$ is the class-conditional probability.

Returns the joint log-probability of the samples for each class in the model. The columns correspond to the classes in sorted order, as they appear in the attribute classes_.

Return log-probability estimates for the test vector X .

Returns the log-probability of the samples for each class in the model. The columns correspond to the classes in sorted order, as they appear in the attribute classes_.

Return probability estimates for the test vector X .

Returns the probability of the samples for each class in the model. The columns correspond to the classes in sorted order, as they appear in the attribute classes_.

Return accuracy on provided data and labels.

In multi-label classification, this is the subset accuracy which is a harsh metric since you require for each sample that each label set be correctly predicted.

Mean accuracy of `self.predict(X)` w.r.t. y .

Configure whether metadata should be requested to be passed to the fit method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

True: metadata is requested, and passed to fit if provided. The request is ignored if metadata is not provided.

False: metadata is not requested and the meta-estimator will not pass it to fit.

None: metadata is not requested, and the meta-estimator will raise an error if the user provides it.

str: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Metadata routing for `sample_weight` parameter in fit.

Set the parameters of this estimator.

The method works on simple estimators as well as on nested objects (such as Pipeline). The latter have parameters of the form `<component>__<parameter>` so that it's possible to update each component of a nested object.

Estimator parameters.

Configure whether metadata should be requested to be passed to the `partial_fit` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config`). Please check the User Guide

on how the routing mechanism works.

The options for each parameter are:

True: metadata is requested, and passed to `partial_fit` if provided. The request is ignored if metadata is not provided.

False: metadata is not requested and the meta-estimator will not pass it to `partial_fit`.

None: metadata is not requested, and the meta-estimator will raise an error if the user provides it.

str: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Metadata routing for `classes` parameter in `partial_fit`.

Metadata routing for `sample_weight` parameter in `partial_fit`.

Configure whether metadata should be requested to be passed to the `score` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

True: metadata is requested, and passed to score if provided. The request is ignored if metadata is not provided.

False: metadata is not requested and the meta-estimator will not pass it to score.

None: metadata is not requested, and the meta-estimator will raise an error if the user provides it.

str: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Metadata routing for `sample_weight` parameter in `score`.

Probability calibration of classifiers

Probability Calibration curves

Comparison of Calibration of Classifiers

Classifier comparison

Plotting Learning Curves and Checking Models' Scalability

© Copyright 2007 - 2025, scikit-learn developers (BSD License).